

Hack-AI-thon Screening Assessment

Submitted by: Tanay Singh

B.Tech Computer Science (AI/ML) | SPSU Udaipur | Batch 2026

GitHub: github.com/tanaysingh0312

Submission Date: May 22, 2026

Section 1: LLM Project

I have built two distinct production-grade RAG systems — one during my internship at Pyrotech Technologies, and one as a personal open-source project. Both are described below in detail.

Project A: Enterprise RAG Document Intelligence System

Pyrotech Technologies — Production Internship Project

Problem Statement

Pyrotech Technologies manages a large corpus of industrial documentation — equipment manuals, scanned PDF reports, image-heavy technical documents, and structured data sheets. Field engineers and operations teams had no reliable way to query this knowledge base semantically. Traditional keyword search failed on domain-specific terminology and provided no cited, context-grounded responses. The requirement was a production-grade system capable of handling heterogeneous document types with minimal hallucination and full local deployment for data privacy.

High-Level Architecture

The system follows a multi-stage pipeline from raw document ingestion to grounded LLM response generation:

Document Sources → **OCR & Layout Detection** → **Semantic Chunking** → **Embeddings** → **Vector Storage** → **Hybrid Retrieval** → **Reranking** → **LLM** → **Response**

KEY COMPONENTS

Stage	Component	Description
Ingestion	Document Loader	Handles PDFs, scanned images, structured manuals
OCR	PaddleOCR	Extracts text from scanned documents with high accuracy
Layout Detection	Unstructured	Identifies document structure — tables, headings, sections
Chunking	Semantic Chunker	Custom chunking with metadata (filename, page, chunk ID)
Embeddings	nomic-embed-text (768D)	Generates high-dimensional semantic vectors locally
Storage	ChromaDB	Persistent vector store with metadata-aware filtering
Retrieval	Hybrid (Vector + BM25)	Combines semantic similarity with keyword-based search

Reranking	MMR + Cross-Encoder	Improves result relevance and ensures diversity
Generation	Mistral / Phi-3 via Ollama	Local LLM inference with context-restricted prompting

Tech Stack

- AI/NLP: Ollama, Mistral, Phi-3, nomic-embed-text
- Document Processing: PaddleOCR, Unstructured
- Search & Retrieval: ChromaDB, BM25, RapidFuzz (fuzzy matching)
- Backend: Python, FastAPI (fully async)
- Infrastructure: Docker, local LLM deployment for data privacy

Key Engineering Challenge

The most difficult and unexpected challenge was handling mixed-modality documents — PDFs that combined scanned pages (image-only), digital text pages, embedded tables, and layout-heavy sections all in a single file. A naive extraction approach would silently discard OCR content or merge table rows incorrectly into broken text blocks.

The solution involved building a two-pass ingestion strategy: the first pass used Unstructured to detect layout elements and classify each page as text-native or image-only; the second pass applied PaddleOCR selectively to image pages while using direct extraction for digital text. Table cells were stitched back into structured markdown before chunking to preserve relational context. Additionally, metadata tags (source file, page number, element type) were injected at chunk level so the retrieval pipeline could filter by document region, not just semantic similarity.

This made the system significantly more robust under real document variance, which is what you actually encounter in an enterprise environment versus a clean demo corpus.

REPOSITORY & DEMO

GitHub: <https://github.com/tanaysingh0312/RAG-AI-document-retrieval> | Contributors: Tanay Singh (AI/Backend)

Project B: Local RAG Document Q&A System (Open Source)

Personal Project — Fully Offline, Privacy-First Document Intelligence

Problem Statement

Most RAG demos require OpenAI API keys, send user data to external servers, and collapse when run offline. The goal here was to build a fully local, production-quality document Q&A system where everything — embeddings, LLM inference, vector storage — runs on the user's own machine without any internet dependency or data leaving the device.

High-Level Architecture & Key Components

The system is decoupled into three distinct layers:

Frontend — React + Vite

- React & Vite for the component layer and fast HMR build pipeline
- TailwindCSS for a glassmorphism deep-space themed UI
- Framer Motion for micro-animations and smooth modal transitions
- Axios for async backend communication; React-Markdown for rendering LLM output

Backend — Python + FastAPI

- FastAPI handles async file uploads and query endpoints
- LangChain orchestrates the full pipeline — splitting, embedding, retrieval, and prompting
- PyMuPDF extracts raw text with page number tracking from uploaded PDFs
- python-docx handles Word document ingestion

AI & Data Layer — Fully Local

- ChromaDB as the persistent vector store, serialized to disk between sessions
- Sentence-Transformers (all-MiniLM-L6-v2) for lightweight, offline embedding generation
- Ollama running Qwen3 locally for final answer generation

Data Pipeline

Phase 1 — Ingestion: A PDF is uploaded → PyMuPDF extracts text with page numbers → LangChain's RecursiveCharacterTextSplitter chunks it into ~1000 character segments with 200-character overlap to avoid mid-sentence breaks → each chunk is embedded using Sentence-Transformers → vectors plus metadata (filename, page number, chunk ID) are stored in ChromaDB.

Phase 2 — Retrieval: The user's question is embedded using the same model → ChromaDB performs similarity search, returning the top-5 most relevant chunks → MMR (Maximal Marginal Relevance) ensures result diversity, avoiding redundant chunks → the question and retrieved chunks are bundled into a system prompt with strict grounding instructions → Qwen3 generates the answer, citing the exact file and page numbers used.

Key Engineering Challenge

The most significant challenge was maintaining embedding model consistency across sessions. On the first ingestion, the Sentence-Transformer model is loaded and used to generate vectors. On subsequent queries — or after a system restart — the same model must be used for query embedding, or the cosine similarity scores become meaningless due to dimensional inconsistency. This is easy to miss because ChromaDB will not throw an error; it will simply return irrelevant chunks silently.

The fix was to pin the embedding model at the collection level inside a configuration file loaded at both ingestion and retrieval time, so the model reference is never assumed or hardcoded in more than one place. This also meant the system could be upgraded to a better embedding model in future by simply re-indexing the collection rather than modifying retrieval logic.

REPOSITORY & DEMO

GitHub: github.com/tanaysingh0312 (README includes full setup instructions, dependencies, and local run guide)

Note: Demo video link to be attached in the Google Form submission.

Section 2: Architecture Design — Document Q&A System (10,000 Documents)

The system needs to handle 10,000 internal documents (PDFs, Word files, Confluence pages), answer natural language questions with cited responses, and go beyond what basic keyword search can provide. Below is the full pipeline design with reasoning for each decision.

1. Chunking Strategy

I would use `RecursiveCharacterTextSplitter` as the default chunker with a chunk size of 512 tokens and a 64-token overlap. The recursive strategy is preferred over fixed character splitting because it respects natural document boundaries — it tries to split on paragraphs first, then sentences, then words — which means chunks are more semantically coherent.

For structured documents like Confluence pages with clear section headers, I would apply a header-aware splitter that treats each section as its own chunk while preserving the header as metadata. Tables in Word and PDF documents would be extracted and chunked separately as structured text (markdown-formatted) to avoid fragmenting relational row data.

Every chunk is tagged with metadata: source filename, document type, section heading, page number, and a chunk index. This metadata is critical for citation generation and for enabling filtered retrieval (e.g., only search HR documents for a policy question).

2. Embedding Model

For a 10,000-document internal corpus, I would use `text-embedding-3-small` (OpenAI) if cloud usage is acceptable, or `nomic-embed-text` (768D) for a fully local deployment. The reasoning:

- `text-embedding-3-small` offers strong semantic accuracy across document types with low cost at scale, and handles domain-mixed corpora well without fine-tuning
- `nomic-embed-text` is the best open-source alternative — 768-dimensional, strong on long-document retrieval benchmarks, runs locally via Ollama, and is what I used in my own RAG systems

I would not use `all-MiniLM-L6-v2` for this scale because while it is fast, its lower dimensionality (384D) loses nuance on domain-specific enterprise content. If latency is critical at scale, ONNX-quantized `nomic-embed-text` is a viable middle ground.

3. Vector Store

I would use Qdrant for production at this scale. Reasons:

- Qdrant supports payload-based filtering natively — retrieval can be scoped to document type, date range, or department without post-filtering
- It supports hybrid search out of the box (dense + sparse vectors in one query), which is critical for this use case
- Horizontal scaling and a REST API make it production-friendly

For a simpler or single-server deployment, ChromaDB with persistent storage works well — it is what I have used in production and it handles this document scale without issue. The tradeoff is less native support for advanced filtering and no distributed mode.

4. Retrieval Strategy

I would implement a hybrid retrieval pipeline combining three mechanisms:

- Dense vector search: semantic similarity via cosine distance on embeddings — handles paraphrased questions and conceptual queries
- Sparse/BM25 search: keyword-based matching — handles exact term lookups, proper nouns, and acronyms that embeddings sometimes miss
- Metadata-filtered retrieval: pre-filter by document type or source before vector search to reduce noise

The final ranked list is produced by a fusion step (Reciprocal Rank Fusion or weighted scoring) followed by MMR (Maximal Marginal Relevance) to ensure the top-k results are diverse — avoiding the case where all retrieved chunks come from the same paragraph of the same document.

Retrieved chunks (top 5–8) are passed to the LLM with a strict grounding prompt: answer only using the provided context, cite the source file and page number for each claim. This is what produces cited answers rather than freeform generation.

5. Handling Partially Answering Documents

This is one of the most important design considerations and a common failure point in naive RAG implementations.

My approach is multi-layered:

- Retrieve more chunks than needed (top 8 instead of top 5) to increase the chance of capturing complementary information spread across different documents
- Use cross-encoder reranking as a second pass — the cross-encoder scores each (question, chunk) pair jointly, which catches cases where a chunk is relevant in context but not by embedding similarity alone
- Explicitly prompt the LLM to synthesize partial information: instruct it to combine information from multiple chunks and flag when no single source fully answers the question
- If the combined context is insufficient, the system returns a partial answer with a clear disclaimer and cites which sources were consulted, rather than hallucinating a complete answer

This last point is particularly important: the system should be honest about the boundary of its knowledge rather than confidently producing an incomplete or fabricated answer.

6. Biggest Failure Mode & Mitigation

Failure Mode: Retrieval Quality Collapse on Long-Tail Queries

The biggest failure mode in this design is not hallucination at the generation stage — it is silent retrieval failure. When a user asks a question that is semantically distant from how the relevant document was written (different vocabulary, different framing, implicit domain knowledge), the embedding-based retrieval returns low-relevance chunks. The LLM then generates a confident-sounding answer based on wrong context, and because no error is raised, the user never knows the retrieval failed.

This is particularly common in enterprise settings where the same concept is described differently in policy documents, technical manuals, and Confluence wikis.

Mitigation Strategy

- Retrieval confidence threshold: compute a similarity score threshold below which the system explicitly tells the user it could not find relevant context, rather than proceeding with low-quality chunks

- Query expansion: before retrieval, use the LLM to generate 2–3 alternative phrasings of the user's question, then retrieve against all variants and take the union. This dramatically improves recall on paraphrased queries
- Hybrid search: BM25 catches exact term matches even when embeddings fail — adding this as a parallel retrieval path provides a safety net
- Evaluation loop: periodically sample production queries, score retrieval quality (MRR, NDCG), and identify failure clusters. This turns a silent failure into a measurable metric that can be monitored and improved
- Chunk-level attribution logging: log which chunks were retrieved for each query. When users flag a wrong answer, the chunk log immediately identifies whether the failure was at retrieval or generation — making debugging significantly faster

Section 3: Debug Challenge

INSUFFICIENT INFORMATION PROVIDED. This assessment is intended exclusively for evaluating a candidate's technical and problem-solving abilities. The code and context supplied are deliberately incomplete and are not intended to be solved using AI-assisted tools or autogenerated responses. Any AI-generated solution may be inaccurate, misleading, or considered a violation of the assessment guidelines.